

Prototypes That Ship: How I Moved Strategyzer's Product Discovery into Code

Sep 26, 2026 · @Olaide

Product designers have always worked one step away from the real product. For most of the discipline's history, our tools were built to describe software rather than run it, and they have done that job remarkably well. Specs, mockups and prototypes carry a huge amount of intent from design to engineering. What they can't fully show is how a design behaves once it's real: how an empty state feels, where a long label breaks a layout, what someone sees when they don't have permission to do what they came to do.

The industry has done serious work to narrow that gap. Design systems, shared tokens, component libraries and increasingly capable prototyping tools have brought design and engineering much closer, and made handoff far more precise. Even so, a meaningful share of what we learn about a design still arrives after it's built, when changes cost the most.

AI tools build on that progress in a new way. A designer can now take an idea to a working version in code within hours, try directions that were once too costly to explore, and put something real in front of users and engineers much earlier. Used with care, they move more of that learning into the design process itself.

At Strategyzer, I saw that as a chance to change how our product team discovers and builds, and I took the lead on making it happen. I started with the foundation, rebuilding FondUI, our product design system, as a Claude skill so anything generated in code would look and behave like our product from the first screen. Then I reworked our process around it.

Discovery at Strategyzer now happens in working code. We test ideas as real flows, settle on a direction faster and find edge cases before development starts, and our engineers carry parts of that code directly into the platform. What follows is how I built it and what it has changed for the team.

What the gap cost a small team

I'm the only designer at Strategyzer, working alongside two developers and a head of product. On a team that size, every design decision carries a cost that's easy to underestimate. Comparing two approaches to a flow meant designing both in detail, wiring them into clickable prototypes and walking the team through them, then waiting for engineering time before anyone saw how either one behaved with real data and real states.

That shaped how much we could explore. Good alternatives were sometimes set aside because they were expensive to validate. Edge cases tended to surface during development, when

changing course took the most effort. And while clickable prototypes served our research well, people engage differently with a simulation of software than with software they can actually use.

None of this was a tooling failure, and Figma remains central to how I think and work. The opportunity was to shorten the distance between an idea and a working version of it, so more of the learning could happen while decisions were still cheap to change.

Teaching Claude our design system

AI tools can already generate interfaces, but not your interface. Ask for a settings page and you get something plausible and generic, with the wrong spacing, the wrong type scale and components your product doesn't have. That's fine for a quick sketch. It won't hold up in real product discovery, and engineering can't build on it.

So the foundation had to be FondUI, our product design system. Using Figma's MCP tooling, I extracted its foundations and components from our Figma library and rebuilt them as a self-contained Claude skill, a package of instructions and assets Claude loads whenever it builds UI for Strategyzer. [\[Add detail on the extraction process and how long it took.\]](#)

The tokens were the straightforward part. Color, typography, spacing, radius and elevation sit in semantic hierarchies, exposed as kebab-case CSS custom properties. [\[Confirm these match the variables engineering uses in the codebase.\]](#)

The rules were where the real design work lay. For every token, the skill says when to use it and when to avoid it. For every component, from buttons and the side navigation to inputs, toggles, radios and checkboxes, it says when to reach for it, when another component fits better, and how it behaves in each state, from hover and focus through to disabled and selected. These are judgment calls I'd normally make on instinct or explain in a design review. Writing them down precisely is what makes Claude's output look like it came from our own Figma file.

[\[Image: the FondUI skill. A component rule excerpt next to the matching Figma library page\]](#)

How I work now

Exploration has become cheap, and that has changed how I approach almost every problem. When I have an idea for a flow, I describe it to Claude, it builds a working version with FondUI, and I review it with the same scrutiny I'd give a developer's build: tightening layouts, rewriting copy, checking every state. When a stakeholder asks what happens if a user does something unexpected, I can usually show them the same day.

The bigger shift is in how many ideas we get to test. Where I used to take one or two directions to high fidelity, I now build several and put them side by side as working flows. The team clicks through each one, sees where it struggles, and we settle on a direction with far more confidence and much less debate.

Edge cases get the same treatment. A working prototype won't let you skip them: the empty state has to render something, the error has to say something useful, and the user who changes their mind halfway through still has to land somewhere sensible. I meet those moments while I'm designing, when they're cheapest to get right and when I can give them proper attention.

Our AI work shows this most clearly. Customers were asking for AI, so I designed Strattie AI, an assistant across the whole platform, and built a working prototype of it with Claude and FondUI in a matter of days. Because it was real software rather than a mockup, it taught us things a static design would have hidden: the value was in the questions a good coach asks before recommending anything, and an open chat box was the wrong tool for people who couldn't yet put their problem into words.

That changed the project. Engineering couldn't build the full assistant in the time we had, so I narrowed it to the one job that mattered most: helping people choose the right playbook. On a call, I showed our CEO and our Head of Product the working prototype alongside my research, and we agreed to ship a three-question Playbook Recommender first, with the full assistant to follow.

Designing the Recommender worked the same way. I tried three shapes for it in code, a pop-up in the middle of the screen, a tall side panel and a small panel in the corner, and compared them as working flows before choosing the corner panel. I also built a tour-style version with a progress bar and a big welcome screen, and set it aside once the working version showed it felt like a one-time tour rather than a tool people return to. Tricky cases surfaced early, while they were cheap to fix, like a user changing an earlier answer, or no playbook matching what they need. The Recommender now says so honestly instead of guessing.

[Image: the Playbook Recommender. The three panel shapes I compared, or the tour-style version next to the shipped tool]

Code that engineering keeps

The largest impact has been on the engineering side, and it's the one I underestimated at first.

Because the prototypes use the same tokens and components as the product, they aren't throwaway work. Our engineers take parts of that code and use it directly in the platform. The Playbook Recommender is the clearest example: I built its front end myself with Claude, and that code went into the product as it was. There was no handover and no rebuild. Our engineer connected it to the data behind it, and it went live. Rather than rebuilding each screen from a design file, the team starts from code that already reflects the layout, spacing and state decisions we've made together.

That has changed how we work together. Handoff used to be a file and a walkthrough, followed by a round of questions. Now it's often a working version we review side by side. With layout and

states already settled, our conversations move sooner to architecture, data and performance, and the design carries through to production more faithfully.

"[Quote from a developer on the team]" — [Name, Role]

Testing with something real

Research has improved in the same way. In interviews and validation sessions, participants use prototypes that behave like the product. They type, filter, change their answers and run into dead ends, and they respond to the experience itself, including moments a simulated flow can't easily cover. Between sessions I can adjust the prototype, so each conversation tests a stronger version than the last.

For the Recommender, that meant about 24 sessions with coaches, customer teams and customers. Because people used the real thing, their feedback changed the design directly: I rewrote the questions so they were easier to understand, and so no one worried about picking the wrong answer.

"[Quote from the head of product]" — [Name, Role]

The impact so far

Across the product team, the results show up in five places.

- **More ideas tested:** three panel shapes and two overall approaches, a tour and a tool, compared as working flows for the Recommender alone. [Add the usual number of directions explored per feature before.]
- **Faster decisions:** direction agreed in [time], down from [time].
- **Edge cases caught in design:** changing an answer and finding no match, both resolved before development started.
- **Code reused in production:** the Playbook Recommender's front end went into the platform as built. The project took about six weeks from research to launch, and the team now works at least twice as fast.
- **Stronger validation:** about 24 research sessions run on working prototypes for the Recommender.

What it doesn't replace

None of this makes Figma or design judgment less important. I still explore visual direction in Figma, and the skill is only as good as the system and rules behind it. When FondUI evolves, the skill has to evolve with it, and keeping the two in step is now part of my job.

Claude doesn't decide what's worth building, either. It's very good at producing a well-formed version of an idea. Deciding which ideas deserve to exist, and which trade-offs are right for our users, is still the core of the work. Code that reaches the platform goes through engineering review, as it should. What has changed is how strong the starting point is.

Where it led

FondUI was my first skill, and it changed how I think about what a design system is for. If a system could be encoded well enough for Claude to design our product with it, other teams could use the same approach for their own work. That idea grew into the marketing, sales and client delivery skills I wrote about in [\[link to "The Only Designer in the Room"\]](#).

For our product team, the gap between an idea and a working version of it has almost closed. We test more ideas, choose between them with more confidence and reach development with fewer open questions. And I spend far less of my time describing what the product should do, and far more of it finding out.

Editor's notes (remove before publishing)

What changed in this version

- The main example is now the Strattie AI and Playbook Recommender work, replacing the enterprise access management example. Every new detail comes from the approved Strattie case study.
- "The user without admin rights" in "How I work now" is now "the user who changes their mind halfway through", to match the new example.
- "Switch roles" in "Testing with something real" is now "change their answers".
- Now filled from the case study: the directions compared, the edge cases, the code that shipped, the research sessions and the change testing drove.

Still open

- Extraction: how you pulled FondUI from Figma, any tools beyond Figma MCP, and roughly how long it took.
- Tokens: confirm the CSS custom properties match what engineering uses in the codebase.
- Faster decisions: how long direction took to agree before, and now.
- Directions before: how many you used to take to high fidelity per feature.
- Quotes: a developer and the head of product.

- Images: the FondUI skill rule next to its Figma page, and the Recommender's compared shapes or tour vs tool.
- "Two developers" (team) vs "our engineer" (on this project): confirm both are right.
- Link to "The Only Designer in the Room" in "Where it led".